

.inputrc: VI experience in the shell

deuterium — 2024-01-28

Canonical HTML: <https://deut-erium.github.io/2024/01/28/inputrc.html>

Abstract

.inputrc: VI experience in the shell, published in deuterium's blog.

Readline's vi editing mode gives Bash a command mode and an insert mode. Configure it in ~/.inputrc.

Toggling modes

Press Esc to enter command mode, where keys move the cursor or edit text. Press i or a to return to insert mode. My configuration marks these modes with C and I in the prompt and changes the cursor shape.

I recommend ;; as an alternative to reaching for Esc. The configuration below binds it to switch from insert mode to command mode.

To type two literal semicolons in insert mode, pause between them so the binding does not trigger.

Useful editing motions in vim

The configuration below adds or adjusts these editing commands.

1. **Repeating the last action or command** - By default, pressing . to repeats the last change made. You can bind !! command of bash to re-input the last run command.
2. **Deleting text** - VIM provides several ways to delete text quickly. For example, dd to easily delete the entire line and D (i.e shift-d) to delete the text in front of the cursor.
3. **Additional groupings** - Additionally other text groupings like forward word w, backward word b, inside something i can be paired with delete d (e.g dw) to quickly delete specific texts groups
4. **Changing stuff** - Even easier, just delete the said stuff and go in insert mode, no need to manually type i with same combinations of the text groups
5. **Finding characters**: Vim allows you to find a character forward (f) or backwards (F) to the cursor. Once you find the character, you can use ; to go to the next occurrence and , to the previous occurrence

NOTE: incremental search with / and ? just like vim is not really supported

My thoroughly commented inputrc

```
# TURN ON VIM (E.G. FOR READLINE)
set editing-mode vi

# SHOW THE VIM MODE IN THE PROMPT (COMMAND OR INSERT)
set show-mode-in-prompt on

# SET THE MODE STRING AND CURSOR TO INDICATE THE VIM MODE
#   FOR THE NUMBER AFTER `\\e[`:
#   0: blinking block
#   1: blinking block (default)
#   2: steady block
#   3: blinking underline
#   4: steady underline
#   5: blinking bar (xterm)
#   6: steady bar (xterm)

# distinguish between command and insert mode
# Prepend I to indicate insert mode and have blinking bar
set vi-ins-mode-string I\\e[5 q\\2
# prepend C to indicate command mode and have a blinking block
set vi-cmd-mode-string C\\e[1 q\\2
set blink-matching-paren on

# sets the readline to display possible completions using different colors
# to indicate filetypes determined from env variable LC_COLORS
set colored-stats on

# Completions listed immediately instead of bell when completing word has more than
one possible completion
set show-all-if-ambiguous on

set completion-ignore-case on
set menu-complete-display-prefix on

# Enables the display of only the first 5 unique characters from a group of file or
directory name suggestions when you utilize tab-completion.
# If multiple entries share a long prefix, they will be condensed using ellipses,
making it easier to locate and choose the desired item.
# Useful in directories containing many similarly named images or documents.
set completion-prefix-display-length 5

# Set the bell-style to be visible only i.e no audio played on command completion
# can also be set to none
set bell-style visible

$if mode=vi
#vi mode settings
    set keymap vi-command

    # go into insert mode, re run last command with !! and press enter
    ".": "i!!\\r"
    "|": "A | "

    # delete rest of the line (vi "D" behavior)
    "D": kill-line
```

```

# change line -> delete then go in insert mode
"C": "Da"
"dw": kill-word
"dd": kill-whole-line
"db": backward-kill-word
# delete and change line
"cc": "ddi"
# change word
"cw": "dwi"
# change backward word
"cb": "dbi"
# vi equivalent of delete all word i.e delete the current word entirely
"daw": "lbdW"
"yaw": "lbyW"

# change all word, delete and edit the current word
"caw": "lbcW"

# delete inner word (word under the cursor without the surrounding whitespaces)
"diw": "lbdw"
# yank inner word
"yiw": "lbyw"
# change inner word
"ciw": "lbcw"

# delete around double quoted string -> delete the text in double quoted strings
and the quotes themselves
# F search backward for a double quote, then delete till first forward search of
double quotes
"da\"": "lF\"df\""

# delete inside double quoted string -> delete the text inside the double quoted
strings but not the quotes
"di\"": "lF\"lmtf\"d`t"
# change inside double quoted string basically delete inside double quoted string
and go in insert mode
"ci\"": "di\"i"

# change around double quoted string
"ca\"": "da\"i"

# delete around single quoted string
"da'": "lF'df'"
"di'": "lF'lmtf'd`t"
"ci'": "di'i"
"ca'": "da'i"

# delete around tilde
"da`": "lF`df`"
"di`": "lF`lmtf`d`t"
"ci`": "di`i"
"ca`": "da`i"

# delete around parenthesis
"da(": "lF(df)"
"di(": "lF(lmtf)d`t"
"ci(": "di(i"
"ca(": "da(i"
"da)": "lF(df)"
"di)": "lF(lmtf)d`t"

```

```

"ci)": "di(i"
"ca)": "da(i"

# delete around curly
"da{": "lF{df}"
"di{": "lF{lmtf}d`t"
"ci{": "di{i"
"ca{": "da{i"
"da}": "lF{df}"
"di}": "lF{lmtf}d`t"
"ci}": "di}i"
"ca}": "da}i"

# delete around square brackets
"da[": "lF[df]"
"di[": "lF[lmtf]d`t"
"ci[": "di[i"
"ca[": "da[i"
"da]": "lF[df]"
"di]": "lF[lmtf]d`t"
"ci]": "di]i"
"ca]": "da]i"

# delete around angled brackets
"da<": "lF<df>"
"di<": "lF<lmtf>d`t"
"ci<": "di<i"
"ca<": "da<i"
"da>": "lF<df>"
"di>": "lF<lmtf>d`t"
"ci>": "di>i"
"ca>": "da>i"

# delete around forward slash
"da/": "lF/df/"
"di/": "lF/lmtf/d`t"
"ci/": "di/i"
"ca/": "da/i"

# delete around colon
"da:": "lF:df:"
"di:": "lF:lmtf:d`t"
"ci:": "di:i"
"ca:": "da:i"

"gg": beginning-of-history
"G" : end-of-history

# backward history search on up arrow
"\e\e[A": history-search-backward
# forward history search on down arrow
"\e\e[B": history-search-forward

# settings to be changed in insert mode
set keymap vi-insert
";;": vi-movement-mode
    # using ;; to easily move to command mode from insert mode instead of going to
press Escape key each time
    # ;; as I find it relatively easily on the home row and it doesnt interfere

```

```
with typing in insert mode
    # as commands to come by with ;; are rare?
    TAB: menu-complete
    # shift tab to menu complete backward
    "\e[Z": menu-complete-backward

# end vi mode settings
$endif
```

Check the latest version [in my dotfiles](#) in an event I forget to update my findings here

How to source `.inputrc`

You can instruct readline to re-read the `.inputrc` file

```
bind -f ~/.inputrc
```

Alternatively you can change the keymap of your preference to `re-read-init-file`

```
# in .inputrc
# press control-x control-r (which should be the default)
"\C-x\C-r": re-read-init-file
```

Editing long commands

Press `v` in command mode to open the current command in your editor. Bash uses `$VISUAL` if set, otherwise `$EDITOR` (shame if it's not set to Vim in your `.bashrc`). Saving and closing the editor executes the edited command.

Caveats

1. Quitting without saving will still execute the original command.

TIP: Adding a “`#`” at the beginning of a command can help prevent its accidental execution!

2. Multiline commands, separated by a backslash (“`\`”), may appear differently in searchable command history due to replacement with spaces. For example, entering `echo hello \n world` becomes `echo hello world` in the history log.
3. Separate entries are created for multiple commands in history search, meaning each command is treated independently, regardless of whether they were entered together or not.

References

- [Bash command line editing manual](#)
- [Usenix bash notes](#)
- [Superuser, how do i reload inputrc](#)

- Inputrc for humans
- The very basics of vim

| Interactive controls are omitted from this PDF. See the canonical HTML version.